

Why does sASM work — and only when the subdomain solve is inexact?

An interpretation through information propagation and overlap over-counting

Minimal 1D / 2D Laplace experiment (all numerics in PETSc): BASIC vs. sASM additive Schwarz, with *exact* (Cholesky) and *inexact* (ICC(0)) subdomain solves.

1D: 256 DOF, 8 subdomains. 2D: 64^2 grid, 4×4 subdomains.

The result that frames everything

CG iterations to $\|r\|/\|b\| < 10^{-8}$, same problem, only the subdomain solver changes

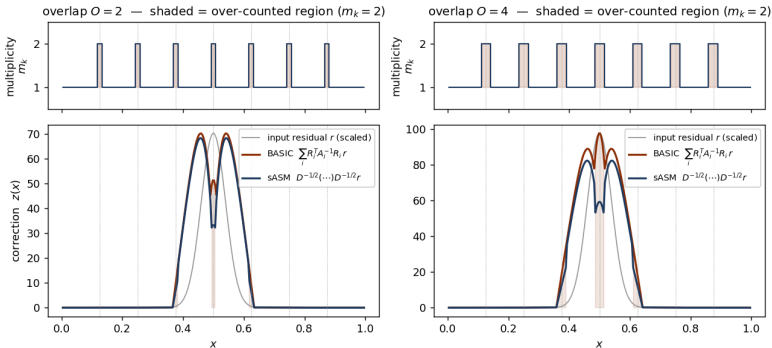
	1D Laplace		2D Laplace	
	exact	ICC(0)	exact	ICC(0)
BASIC	15	15	23	102
sASM	25	25	27	67

- ▶ **sASM beats BASIC only in one cell: 2D with an inexact solve (67 vs 102).**
Everywhere else sASM $\approx$ or slightly worse than BASIC.
- ▶ 1D “ICC” = exact: a tridiagonal matrix has no fill, so ICC(0) is the complete Cholesky.
Hence 1D cannot show the anomaly at all.
- ▶ **Question:** what makes the *inexact* solve bad, and why does sASM fix it precisely there?

Mechanism, part 1: BASIC over-counts the overlap

one preconditioner apply $\sum_i R_i^\top A_i^{-1} R_i r$ to a localized residual

Fig. 1 — One preconditioner apply: BASIC over-counts the overlap; sASM normalizes it

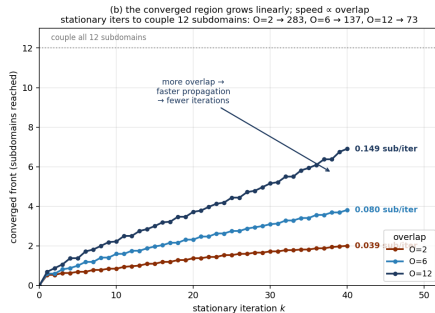
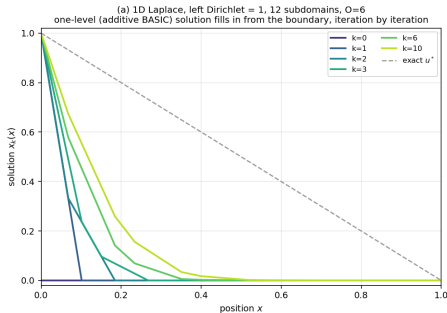


- In the overlap a DOF belongs to m_k subdomains, so its correction is *added* m_k times.
- **BASIC** (red) bumps up at every overlap seam; **sASM** (blue) is normalized by $1/\sqrt{m_k}$.
- This over-count is the algebraic fact $\sum_i R_i^\top R_i = \mathbf{D} = \text{diag}(m_k)$. It is harmless *by itself* — the next slides show when it bites.

Mechanism, part 2: information propagation — the one-level latency

Laplace is globally coupled (A sparse, A^{-1} dense), but each Schwarz step solves only *local* problems

0 — Information propagation, quantified: one-level Schwarz couples the domain at finite speed ($\propto$ overlap); that latency = the small $\lambda_{\min}$ a coarse space ren



- ▶ (a) the solution *crawls in* from the boundary; (b) the converged front advances at **finite speed, growing with overlap** (0.039/0.080/0.149 sub/iter for $O=2/6/12$) — additive Schwarz, slower than the “1 subdomain/iter” multiplicative slogan.
- ▶ So one-level coupling costs iterations that **grow with #subdomains** — this latency is the small $\lambda_{\min}$ (slow global mode); a coarse space transfers it globally in one step (Fig. 9).

Q1 — Why is the inexact (ICC) solve bad?

the abstract Schwarz bound: the two factors *multiply*

$$\kappa(\mathbf{M}^{-1}\mathbf{A}) \leq C_0^2 \cdot \underbrace{\omega}_{\substack{\text{(A) inexactness} \\ \text{exact}=1, \text{ ICC}>1}} \cdot \underbrace{N_c}_{\substack{\text{(B) over-count} \\ \hat{N}=\max_k m_k \leq N_c}}$$

- ▶ Over-count factor: $\mathbf{D} = \text{diag}(m_k)$ measures the **max multiplicity** $\hat{N}$; rigorously $\hat{N} \leq N_c$ (colour count), Gander et al. 2015, Rem. 2.8. It is what the scaling removes.
- ▶ **Exact** ($\omega=1$): each local correction is the *exact* local solution; over-counting only lifts $\lambda_{\max} \leq \hat{N}$ — bounded, CG tolerates it. *No anomaly* (table: 23 vs 27).
- ▶ **ICC** ($\omega>1$): each correction carries an *error*; BASIC **adds** m_k **of them** in the overlap. They accumulate: $\lambda_{\max} \approx \omega \hat{N}$ — inexactness *multiplied* by the over-count.
- ▶ Wider overlap: fixed-fill ICC is a worse approximation ($\omega \uparrow$) *and* $\hat{N} \uparrow$ — so κ *rises* with overlap (the anomaly). The amplified error **piles up at the seams**.

Q1 — the evidence

Fig. 3 — 2D Laplace: sASM beats BASIC ONLY with an inexact solve
(exact: 23 vs 27; ICC(0): 67 vs 102) — over-counting bites only when coupled with inexact

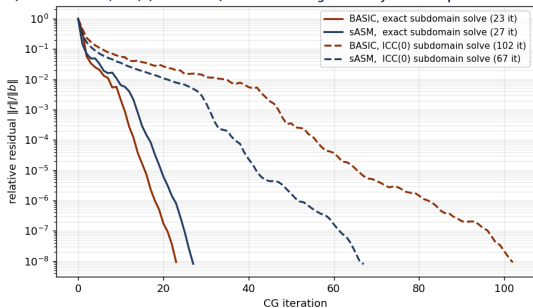
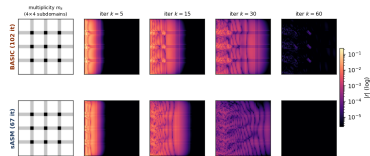


Fig. 4 — ICC(0) subdomain solve: residual $|r|$ at MATCHED CG iterations. BASIC's residual lingers at the overlap seams; sASM clears it → 67 vs 102 iters



ICC residual field at matched CG iterations: at $k=60$ BASIC is still bright (error not cleared), the seams lit up by the over-counted ICC error.

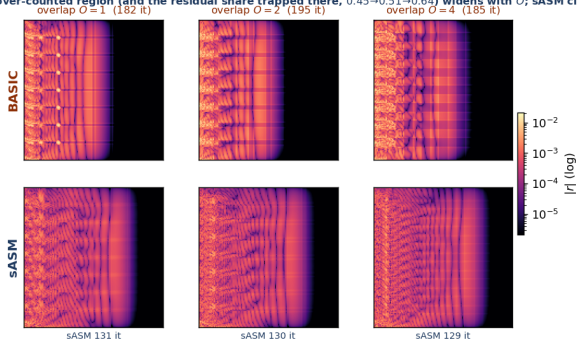
Residual history. Exact: BASIC $\approx$ sASM (both fast). ICC:
BASIC 102 blows up.

“Inexact is bad” = the local error is **counted** m_k **times in the overlap** ($A \times B$), accumulating into seam residual that costs iterations — and worsens as overlap grows.

Q1, deeper — the seam error grows with overlap

2D Laplace, 128^2 , 8×8 subdomains, ICC(0); residual $|r|$ at fixed iteration $k=40$

Fig. 5 — Overlap sweep (128^2 , 8×8 , ICC(0), residual at fixed $k=40$): BASIC's residual sits in the over-counted seams; the over-counted region (and the residual share trapped there, $0.45 \rightarrow 0.51 \rightarrow 0.64$) widens with O ; sASM clears it at every O

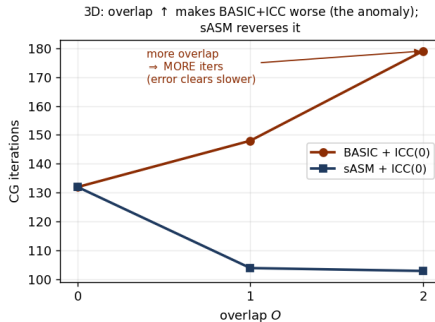
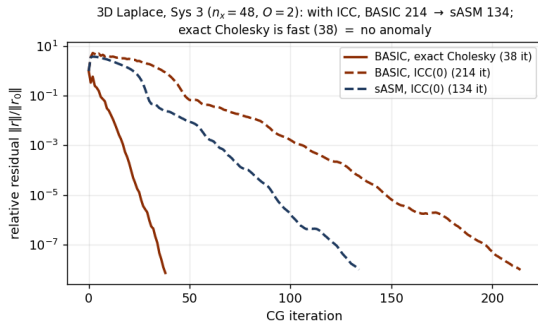


- The over-counted seam region *widens* with overlap, and the residual share trapped there grows $0.45 \rightarrow 0.51 \rightarrow 0.64$.
- sASM is better at *every* overlap (iters BASIC 182/195/185 vs sASM 131/130/129).
- Honest note: in 2D the iteration count vs O is non-monotone (ICC is only mildly inexact in 2D). The clean monotone “overlap $\uparrow \Rightarrow$ iter $\uparrow$ ” is a 3D effect $\rightarrow$ next slide.

The same effect on the real 3D problem (Sys 3)

mixed Dirichlet/Neumann Laplace, $n_x=48$, 4 ranks — the actual torso-type solve

Fig. 6 — The same effect on the real 3D Sys 3 (mixed Dirichlet/Neumann Laplace)



- ▶ **Left:** ICC — BASIC 214 slow, sASM 134 recovers; exact Cholesky 38 (no anomaly, the reference).
- ▶ **Right:** the clean monotone anomaly — BASIC+ICC 132 $\rightarrow$ 148 $\rightarrow$ 179 (more overlap, more iters = error clears slower); sASM 132 $\rightarrow$ 104 $\rightarrow$ 103 reverses it.

Q2 — Why does sASM overcome it?

$$\mathbf{M}_{\text{sASM}}^{-1} = \mathbf{D}^{-1/2} \left(\sum_i R_i^\top A_i^{-1} R_i \right) \mathbf{D}^{-1/2}, \text{ partition of unity } \sum_i d_i^2 = 1$$

$$\text{BASIC: } \lambda_{\max} \leq \omega \hat{N} \quad \xrightarrow{\mathbf{D}^{-1/2}(\cdot)\mathbf{D}^{-1/2}} \quad \text{sASM: } \lambda_{\max} \leq \omega$$

- ▶ The $\mathbf{D}^{-1/2}$ sandwich restores a partition of unity ($\sum_i \text{scaled } R_i^\top R_i = I$), so each DOF is counted **once**: the multiplicity over-count $\hat{N}$ is **removed** (Fig. 1 blue flat; Fig. 8 sASM flat).
- ▶ **This breaks the multiplication.** ω is no longer multiplied by $\hat{N}$: the ICC error in the overlap is counted once, *not accumulated* $\Rightarrow$ no seam pile-up.
- ▶ Result (ICC): seam residual clears (Fig. 4, sASM nearly dark at $k=60$); CG recovers **102** $\rightarrow$ **67**. Measured: $\lambda_{\max}(\text{sASM}, \text{ICC}) \approx 1.25 = \omega$, flat in overlap (Fig. 8a).
- ▶ **sASM does not fix the inexactness** (ω still > 1) — it stops the over-count from *amplifying* it. So $\kappa \lesssim C_0^2 \omega$, and overlap helps again.

What does “exact / inexact subdomain solve” actually solve?

$A_i = R_i A R_i^\top$ — the global Laplace restricted to subdomain i (with overlap)

- ▶ A_i implicitly imposes **homogeneous Dirichlet** on the *artificial cut* between subdomains (the cross-interface coupling is dropped), and keeps the physical BC on faces that touch the real boundary.
- ▶ **Exact solve** A_i^{-1} (Cholesky) = *exactly* solving a local Dirichlet–Laplace subproblem.
 $\omega = 1$.
- ▶ **Inexact solve** ICC(0) = exactly solving a *perturbed* operator $\tilde{A}_i = A_i - E_i$, where E_i is the fill that ICC(0) *drops*. Equivalently, the same Dirichlet subproblem solved only approximately. This E_i **is** the source of $\omega > 1$, and its error sits in the overlap/seam.

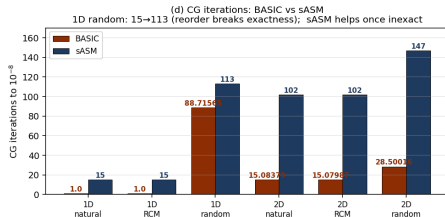
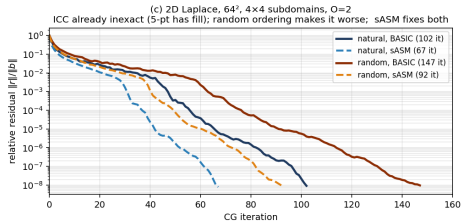
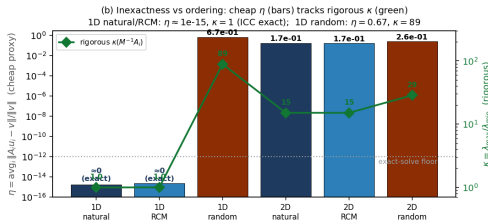
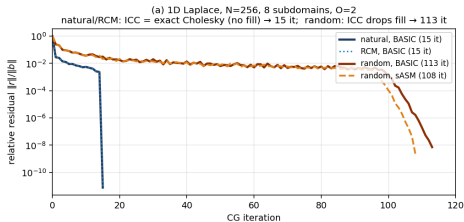
So “inexact” is not mysterious

ICC(0) keeps only A_i ’s nonzero pattern and throws away all fill-in. Whether that loses anything depends entirely on the **ordering** of the unknowns — next slide.

Reordering the unknowns turns the ICC inexact

only the ICC *ordering* changes (natural / RCM / random); subdomains, overlap, weights fixed

Fig. 7 — Reordering the subdomain unknowns turns the ICC factorization inexact (Q1: 1D natural ICC = exact; random ICC $\neq$ exact)



Reading Fig. 7 — reordering breaks (or keeps) exactness

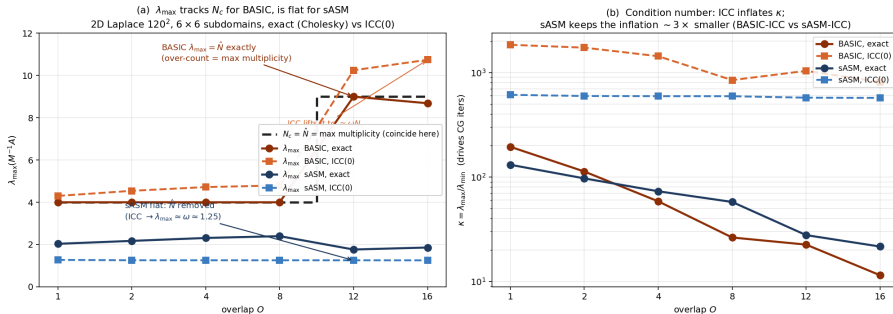
η (cheap ℓ_2 residual, a yes/no exact-flag) and the rigorous $\kappa(M_i^{-1}A_i) = \lambda_{\max}/\lambda_{\min}$ track together

- ▶ **(b) the headline.** 1D *natural* & RCM: $\eta \approx 10^{-15}$, $\kappa = 1$ — ICC is exact Cholesky (a tridiagonal has *no fill*, so dropping fill drops nothing). 1D *random*: $\eta = 0.67$, $\kappa = 89$ — the permutation creates fill that ICC(0) throws away, so ICC is now inexact. (η is only a flag; κ , the green line, is the rigorous condition number.)
- ▶ **(a)/(d) 1D iterations.** natural/RCM converge in 15; random blows up to 113. RCM keeps the 1D problem banded, so it stays exact — *not every* permutation breaks exactness, only a band-widening (random) one.
- ▶ **(c)/(d) 2D.** The 5-point ICC already has fill ($\eta=0.17$); random makes it worse ($\eta=0.26$): $102 \rightarrow 147$ it. **sASM repairs both** ($102 \rightarrow 67$, $147 \rightarrow 92$), more so the more inexact the solve.

Measuring the over-count directly: $\lambda_{\max}$ of $M^{-1}A$

2D Laplace 120^2 , 6×6 subdomains; $\lambda_{\max}/\lambda_{\min}$ from KSPCG (PCShELL wrapping BASIC/sASM)

Fig. 8 — Measuring the over-count: $\lambda_{\max}(\text{BASIC, exact}) = \hat{N}$ (max multiplicity $\leq N_c$); sASM's $D^{-1/2}$ removes it ($\kappa \leq C_0^2 \omega N_c$)



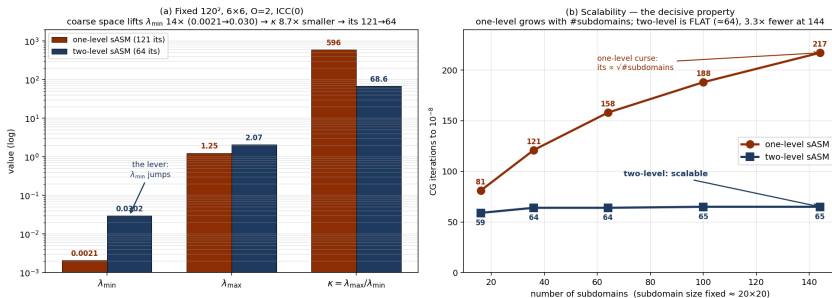
- (a) $\lambda_{\max}(\text{BASIC, exact}) = \hat{N}$ exactly (4, then 9 when overlap reaches the 2nd neighbour). ICC lifts it to $\approx \omega \hat{N}$. sASM is flat ($\approx \omega = 1.25$ for ICC): the multiplicity $\hat{N}$ is removed — the $\omega \times \hat{N}$ product is broken.
- (b) ICC inflates $\kappa \sim 5\text{--}10\times$; sASM-ICC keeps it $\sim 3\times$ below BASIC-ICC $\Rightarrow \sim \sqrt{3}$ fewer CG iters. ($\hat{N} \leq N_c$; they coincide in this regular layout.)

Improving sASM: the bottleneck is now $\lambda_{\min}$ — add a coarse space

sASM fixed $\lambda_{\max} \approx \omega$; what is left is tiny $\lambda_{\min} \geq C_0^{-2}$ (missing global coupling) $\Rightarrow$ two-level sASM

$$\mathbf{M}_2^{-1} = \underbrace{R_0^\top A_0^{-1} R_0}_{\text{coarse: lifts } \lambda_{\min}} + \underbrace{\mathbf{D}^{-1/2} \left(\sum_i R_i^\top A_i^{-1} R_i \right) \mathbf{D}^{-1/2}}_{\text{sASM fine: keeps } \lambda_{\max} \approx \omega} \quad (\text{Nicolaides PoU, symmetric, CG-OK})$$

Fig. 9 — Improving sASM: add a coarse space. The bottleneck is $\lambda_{\min}$ (missing global coupling), not $\lambda_{\max}$; a two-level sASM fixes it and is scalable.



(a) $\lambda_{\min}$ $14 \times$ up, κ $8.7 \times$ down, its $121 \rightarrow 64$. (b) one-level $\rightarrow 217$; two-level flat (≈ 64) = scalable.

Conclusion

The one-sentence picture

BASIC counts the overlap twice (Fig. 1). With *exact* solves that is harmless — it only lifts $\lambda_{\max}$, which CG tolerates (Fig. 3, solid). With an *inexact* ICC solve the local error is counted m_k times, so “inexactness $\times$ over-count” accumulates at the seams (Fig. 4) and convergence degrades (Fig. 3, dashed, 102). sASM’s $\mathbf{D}^{-1/2}$ partition of unity removes the multiplicity over-count, breaking that product, so the inexact corrections are no longer amplified and CG recovers (102 $\rightarrow$ 67).

- ▶ **Q1:** inexactness is bad because BASIC *multiplies* it by the multiplicity over-count $\hat{N} = \max_k m_k (\leq N_c)$; both grow with overlap, so iterations rise with overlap. Measured: $\lambda_{\max}(\text{BASIC,exact}) = \hat{N}$, lifted to $\omega\hat{N}$ by ICC (Fig. 8).
- ▶ **Q2:** sASM removes $\hat{N}$ (partition of unity, Fig. 8 flat), so the inexactness is no longer amplified — it helps *exactly* when the solve is inexact, and is neutral when exact.
- ▶ Information propagates at a finite, overlap-set speed (Fig. 2/10): a one-level latency that grows with #subdomains, untouched by *weighting*.
- ▶ **To reduce iterations a lot:** the residual bottleneck is now $\lambda_{\min}$, not $\lambda_{\max}$ — add a coarse space. A symmetric **two-level sASM** halves iterations and makes them *scalable* (flat in #subdomains, Fig. 9).